

Loan Default Prediction via Logistic Regression

Cameron Batts

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.1      ✓ stringr    1.5.2
## ✓ ggplot2    4.0.3      ✓ tibble     3.3.0
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.2.1
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readr)
```

```
loanData = readRDS("LoanData.rds")
```

```
glimpse(loanData)
```

```
## Rows: 29,092
## Columns: 8
## $ isLoanDefault    <int> 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, ...
## $ loanAmount       <int> 5000, 2400, 10000, 5000, 3000, 12000, 9000, 3000, 1000...
## $ interestRate     <dbl> 10.65, NA, 13.49, NA, NA, 12.69, 13.49, 9.91, 10.65, 1...
## $ creditGrade       <fct> B, C, C, A, E, B, C, B, B, D, C, A, B, A, B, B, B, ...
## $ employmentYears  <int> 10, 25, 13, 3, 9, 11, 0, 3, 3, 0, 4, 13, 1, 6, 17, 13,...
## $ homeLiving        <fct> RENT, RENT, RENT, RENT, RENT, OWN, RENT, RENT, RENT, R...
## $ incomeAnnual      <dbl> 24000.00, 12252.00, 49200.00, 36000.00, 48000.00, 7500...
## $ ageYears          <int> 33, 31, 24, 39, 24, 28, 22, 22, 28, 22, 23, 27, 30, 24...
```

```
head(loanData)
```

```
##      isLoanDefault loanAmount interestRate creditGrade employmentYears homeLiving
## 1              0       5000       10.65          B           10         RENT
## 2              0       2400          NA          C           25         RENT
## 3              0      10000      13.49          C           13         RENT
## 4              0       5000          NA          A            3         RENT
## 5              0       3000          NA          E            9         RENT
## 6              0      12000      12.69          B           11         OWN
##      incomeAnnual ageYears
## 1          24000       33
## 2          12252       31
## 3          49200       24
## 4          36000       39
## 5          48000       24
## 6          75000       28
```

```
# homeLiving indicates whether the borrower rents or owns a home
# creditGrade is the credit risk rating grade assigned to the borrower
```

```
library(gmodels)
CrossTable(loanData$creditGrade, loanData$isLoanDefault)
```

```
##
##
##      Cell Contents
## |-----|
## |                      N |
## | Chi-square contribution |
## |      N / Row Total |
## |      N / Col Total |
## |      N / Table Total |
## |-----|
##
##
## Total Observations in Table:  29092
##
##
##      | loanData$isLoanDefault
## loanData$creditGrade |      0 |      1 | Row Total |
## -----|-----|-----|-----|
##           A |    9084 |    565 |    9649 |
##           |    29.764 |    238.561 |
##           |    0.941 |    0.059 |    0.332 |
##           |    0.351 |    0.175 |
##           |    0.312 |    0.019 |
```

##				
##	B	8344	985	9329
##		0.299	2.398	
##		0.894	0.106	0.321
##		0.323	0.305	
##		0.287	0.034	
##				
##	C	4904	844	5748
##		8.337	66.821	
##		0.853	0.147	0.198
##		0.190	0.262	
##		0.169	0.029	
##				
##	D	2651	580	3231
##		17.096	137.024	
##		0.820	0.180	0.111
##		0.102	0.180	
##		0.091	0.020	
##				
##	E	692	176	868
##		8.235	66.004	
##		0.797	0.203	0.030
##		0.027	0.055	
##		0.024	0.006	
##				
##	F	155	56	211
##		5.663	45.394	
##		0.735	0.265	0.007
##		0.006	0.017	
##		0.005	0.002	
##				
##	G	35	21	56
##		4.392	35.206	
##		0.625	0.375	0.002
##		0.001	0.007	
##		0.001	0.001	
##				
##	Column Total	25865	3227	29092
##		0.889	0.111	
##				
##				
##				

A appears to be the best credit rating. The default rate for A is the lowest among all credit grades. As the grade moves from A to G, the proportion of borrowers who default increases, confirming that A represents the most creditworthy group.

```
library(ggplot2)
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
```

```
## The following object is masked from 'package:dplyr':
##
##      combine
```

```
summary(loanData[, c("employmentYears", "incomeAnnual", "ageYears")])
```

```
##  employmentYears  incomeAnnual    ageYears
##  Min.   : 0.000    Min.   : 4000    Min.   : 20.0
##  1st Qu.: 2.000    1st Qu.: 40000    1st Qu.: 23.0
##  Median : 4.000    Median : 56424    Median : 26.0
##  Mean   : 6.145    Mean   : 67169    Mean   : 27.7
##  3rd Qu.: 8.000    3rd Qu.: 80000    3rd Qu.: 30.0
##  Max.   :62.000    Max.   :6000000    Max.   :144.0
##  NA's   :809
```

```
pIncomeAge = ggplot(loanData, aes(x = ageYears, y = incomeAnnual, color = factor(isLoanDefault))) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(x = "Age (Years)", y = "Income (Annual)", title = "Income vs Age")
```

```
pIncomeEmployment = ggplot(loanData, aes(x = employmentYears, y = incomeAnnual, color = factor(isLoanDefault))) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(x = "Employment Years", y = "Income (Annual)", title = "Income vs Employment")
```

```
pEmploymentAge = ggplot(loanData, aes(x = ageYears, y = employmentYears, color = factor(isLoanDefault))) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(x = "Age (Years)", y = "Employment Years", title = "Employment vs Age")
```

```
grid.arrange(pIncomeAge, pIncomeEmployment, pEmploymentAge, nrow = 2)
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 809 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 809 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 809 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 809 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



```
loanData[!is.na(loanData$ageYears) & (1000 < loanData$ageYears), ]
```

```
## [1] isLoanDefault    loanAmount      interestRate    creditGrade
## [5] employmentYears  homeLiving      incomeAnnual    ageYears
## <0 rows> (or 0-length row.names)
```

```
loanDataNoOutliers = loanData[-19486, ]
```

```
str(loanData)
```

```
## 'data.frame':    29092 obs. of  8 variables:
## $ isLoanDefault : int  0 0 0 0 0 0 1 0 1 0 ...
## $ loanAmount : int  5000 2400 10000 5000 3000 12000 9000 3000 10000 1000 ...
## $ interestRate : num  10.7 NA 13.5 NA NA ...
## $ creditGrade : Factor w/ 7 levels "A","B","C","D",...: 2 3 3 1 5 2 3 2 2 4 ...
## $ employmentYears: int  10 25 13 3 9 11 0 3 3 0 ...
## $ homeLiving : Factor w/ 4 levels "MORTGAGE","OTHER",...: 4 4 4 4 4 3 4 4 4 4
## ...
## $ incomeAnnual : num  24000 12252 49200 36000 48000 ...
## $ ageYears : int  33 31 24 39 24 28 22 22 28 22 ...
```

```
str(loanDataNoOutliers)
```

```
## 'data.frame':    29091 obs. of  8 variables:
## $ isLoanDefault : int  0 0 0 0 0 0 1 0 1 0 ...
## $ loanAmount : int  5000 2400 10000 5000 3000 12000 9000 3000 10000 1000 ...
## $ interestRate : num  10.7 NA 13.5 NA NA ...
## $ creditGrade : Factor w/ 7 levels "A","B","C","D",...: 2 3 3 1 5 2 3 2 2 4 ...
## $ employmentYears: int  10 25 13 3 9 11 0 3 3 0 ...
## $ homeLiving : Factor w/ 4 levels "MORTGAGE","OTHER",...: 4 4 4 4 4 3 4 4 4 4
## ...
## $ incomeAnnual : num  24000 12252 49200 36000 48000 ...
## $ ageYears : int  33 31 24 39 24 28 22 22 28 22 ...
```

```
nrow(loanData)
```

```
## [1] 29092
```

```
nrow(loanDataNoOutliers)
```

```
## [1] 29091
```

A copy is maintained so the original data can be referenced if needed. In cases where the dataset is very large and storage is expensive, or where there are many intermediate steps, it may make sense to skip intermediate copies and only save when major changes are made.

```
loanDataNoOutliersNA = loanDataNoOutliers
```

```
na_percent = sapply(loanDataNoOutliersNA, function(x) mean(is.na(x)) * 100)
na_percent
```

##	isLoanDefault	loanAmount	interestRate	creditGrade	employmentYears
##	0.000000	0.000000	9.542470	0.000000	2.780929
##	homeLiving	incomeAnnual	ageYears		
##	0.000000	0.000000	0.000000		

```
for (colname in names(loanDataNoOutliersNA)) {
  if (any(is.na(loanDataNoOutliersNA[[colname]]))) {
    indicator_name = paste0(colname, "_missing")
    loanDataNoOutliersNA[[indicator_name]] =
      ifelse(is.na(loanDataNoOutliersNA[[colname]]), 1, 0)
  }
}
```

```
summary(loanDataNoOutliersNA)
```

```
## isLoanDefault      loanAmount      interestRate      creditGrade      employmentYears
## Min.      :0.0000    Min.      : 500    Min.      : 5.42    A:9649      Min.      : 0.000
## 1st Qu.:0.0000    1st Qu.: 5000    1st Qu.: 7.90    B:9329      1st Qu.: 2.000
## Median :0.0000    Median : 8000    Median :10.99    C:5747      Median : 4.000
## Mean   :0.1109    Mean   : 9594    Mean   :11.00    D:3231      Mean   : 6.145
## 3rd Qu.:0.0000    3rd Qu.:12250    3rd Qu.:13.47    E: 868      3rd Qu.: 8.000
## Max.   :1.0000    Max.   :35000    Max.   :23.22    F: 211      Max.   :62.000
##                                     NA's   :2776    G: 56      NA's   :809
##      homeLiving      incomeAnnual      ageYears      interestRate_missing
## MORTGAGE:12001    Min.      : 4000    Min.      :20.0    Min.      :0.00000
## OTHER      : 97    1st Qu.: 40000    1st Qu.:23.0    1st Qu.:0.00000
## OWN        : 2301    Median : 56400    Median :26.0    Median :0.00000
## RENT       :14692    Mean   : 66965    Mean   :27.7    Mean   :0.09542
##                                     3rd Qu.: 80000    3rd Qu.:30.0    3rd Qu.:0.00000
##                                     Max.   :2039784    Max.   :94.0    Max.   :1.00000
##
##      employmentYears_missing
## Min.      :0.00000
## 1st Qu.:0.00000
## Median :0.00000
## Mean   :0.02781
## 3rd Qu.:0.00000
## Max.   :1.00000
##
```

Other variables to consider for categorization include incomeAnnual, where higher income can indicate lower default risk, and ageYears, where very young or very old borrowers may carry higher risk.

```
set.seed(2020)

loanTestIndices = sample(1:nrow(loanDataNoOutliersNA),
                        size = nrow(loanDataNoOutliersNA) / 3)

loanTest = loanDataNoOutliersNA[loanTestIndices, ]
loanTrain = loanDataNoOutliersNA[-loanTestIndices, ]

str(loanTrain)
```



```
## 'data.frame': 19394 obs. of 10 variables:
## $ isLoanDefault : int 0 0 0 1 0 1 0 0 0 0 ...
## $ loanAmount : int 5000 2400 10000 9000 3000 10000 10000 6000 10000
10000 ...
## $ interestRate : num 10.65 NA 13.49 13.49 9.91 ...
## $ creditGrade : Factor w/ 7 levels "A","B","C","D",...: 2 3 3 3 2 2 3 2
2 2 ...
## $ employmentYears : int 10 25 13 0 3 3 4 1 13 5 ...
## $ homeLiving : Factor w/ 4 levels "MORTGAGE","OTHER",...: 4 4 4 4 4 4
4 1 3 4 ...
## $ incomeAnnual : num 24000 12252 49200 30000 15000 ...
## $ ageYears : int 33 31 24 22 22 28 23 30 23 22 ...
## $ interestRate_missing : num 0 1 0 0 0 0 1 1 0 0 ...
## $ employmentYears_missing: num 0 0 0 0 0 0 0 0 0 0 ...
```

```
str(loanTest)
```

```
## 'data.frame': 9697 obs. of 10 variables:
## $ isLoanDefault : int 0 0 0 0 0 0 0 0 0 0 ...
## $ loanAmount : int 6700 14900 14000 10000 12000 9450 6000 7400 4600
8500 ...
## $ interestRate : num 11.99 9.63 NA 14.79 11.71 ...
## $ creditGrade : Factor w/ 7 levels "A","B","C","D",...: 2 1 2 3 2 2 3 3
2 1 ...
## $ employmentYears : int 3 1 13 1 9 9 2 0 1 21 ...
## $ homeLiving : Factor w/ 4 levels "MORTGAGE","OTHER",...: 4 4 1 1 4 1
4 3 4 1 ...
## $ incomeAnnual : num 48000 63000 310000 100000 38340 ...
## $ ageYears : int 22 37 30 29 25 35 38 26 23 24 ...
## $ interestRate_missing : num 0 0 1 0 0 0 0 1 0 0 ...
## $ employmentYears_missing: num 0 0 0 0 0 0 0 0 0 0 ...
```

```
glmBase = glm(isLoanDefault ~ .,
              family = "binomial",
              data = loanTrain)

summary(glmBase)
```

```
##
## Call:
## glm(formula = isLoanDefault ~ ., family = "binomial", data = loanTrain)
##
## Coefficients: (2 not defined because of singularities)
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -2.912e+00  2.303e-01 -12.642  < 2e-16 ***
## loanAmount    -1.706e-06  4.487e-06  -0.380  0.703901
## interestRate   7.287e-02  2.476e-02   2.943  0.003250 **
## creditGradeB   3.873e-01  1.165e-01   3.324  0.000888 ***
## creditGradeC   6.275e-01  1.691e-01   3.710  0.000207 ***
## creditGradeD   7.282e-01  2.144e-01   3.396  0.000684 ***
## creditGradeE   7.762e-01  2.680e-01   2.896  0.003778 **
## creditGradeF   1.249e+00  3.534e-01   3.533  0.000410 ***
## creditGradeG   1.229e+00  4.889e-01   2.515  0.011915 *
## employmentYears 9.760e-03  3.800e-03   2.569  0.010210 *
## homeLivingOTHER 5.424e-01  3.787e-01   1.432  0.152107
## homeLivingOWN  -1.317e-01  1.023e-01  -1.288  0.197627
## homeLivingRENT  4.611e-03  5.721e-02   0.081  0.935754
## incomeAnnual   -5.894e-06  8.512e-07  -6.924  4.38e-12 ***
## ageYears       -4.822e-03  4.219e-03  -1.143  0.252988
## interestRate_missing      NA          NA      NA      NA
## employmentYears_missing    NA          NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 11658  on 17009  degrees of freedom
## Residual deviance: 11170  on 16995  degrees of freedom
## (2384 observations deleted due to missingness)
## AIC: 11200
##
## Number of Fisher Scoring iterations: 5
```

interestRate has a p-value of .003 (highly significant) and all creditGrade levels are highly significant as shown by the asterisk markers.

*# Most significant: interestRate, all creditGrade levels (B-G), incomeAnnual
Mildly significant: employmentYears
Not significant: loanAmount, all homeLiving categories, ageYears*

```
predictionsBase = predict(
  object = glmBase,
  newdata = loanTest,
  type = "response"
)

loanTest$predictionsBase = predictionsBase

summary(predictionsBase)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
## 0.00003 0.06169 0.10240 0.10831 0.14452 0.42429   1137
```

```
summary(loanTest[, "isLoanDefault"])
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000 0.0000 0.0000 0.1121 0.0000 1.0000
```

```
summary(predictionsBase - loanTest[, "isLoanDefault"])
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   NA's
## -0.99765 0.05358 0.08909 -0.00290 0.13147 0.42429   1137
```

Q16.

The minimum difference is about -0.9976, meaning the model predicted a very low probability even though the borrower actually defaulted. The median is about 0.089, showing predictions are only slightly above true values for most cases. The mean is near zero and the max of about 0.4243 shows the largest over-prediction is still below 0.5. Overall the model predictions are fairly close to actual outcomes.

```
sum(predictionsBase >= 0.5, na.rm = TRUE)
```

```
## [1] 0
```

```
sum(predictionsBase < 0.5, na.rm = TRUE)
```

```
## [1] 8560
```

With a cutoff of 0.5, almost all predictions are classified as 0 because the model rarely predicts probabilities above 0.5. This means most actual defaults would be missed. Lowering the cutoff to around 0.1-0.2 would increase the number of predicted defaults and improve the model's ability to catch bad loans.

```
sortedPredictions = sort(predictionsBase, decreasing = TRUE)
cutoffIndex = as.integer(0.1121 * length(sortedPredictions))
cutoff = sortedPredictions[cutoffIndex]

isPrediction = ifelse(predictionsBase >= cutoff, 1, 0)

loanTest = cbind(isPrediction, loanTest)

summary(loanTest)
```

```
##      isPrediction  isLoanDefault      loanAmount      interestRate  creditGrade
## Min.      :0.000    Min.      :0.0000    Min.      : 500    Min.      : 5.42    A:3226
## 1st Qu.:0.000    1st Qu.:0.0000    1st Qu.: 5000    1st Qu.: 7.90    B:3061
## Median :0.000    Median :0.0000    Median : 8000    Median :10.99    C:1935
## Mean   :0.112    Mean   :0.1121    Mean   : 9618    Mean   :11.02    D:1103
## 3rd Qu.:0.000    3rd Qu.:0.0000    3rd Qu.:12375    3rd Qu.:13.48    E: 275
## Max.    :1.000    Max.    :1.0000    Max.    :35000    Max.    :23.22    F: 78
## NA's    :1137                                NA's    :919    G: 19
## employmentYears  homeLiving      incomeAnnual      ageYears
## Min.      : 0.000    MORTGAGE:4003    Min.      : 4000    Min.      :20.00
## 1st Qu.: 2.000    OTHER      : 40    1st Qu.: 40000    1st Qu.:23.00
## Median : 4.000    OWN       : 708    Median : 57000    Median :26.00
## Mean   : 6.097    RENT      :4946    Mean   : 67621    Mean   :27.76
## 3rd Qu.: 8.000                                3rd Qu.: 80300    3rd Qu.:30.00
## Max.    :56.000                                Max.    :1900000    Max.    :76.00
## NA's    :239
## interestRate_missing employmentYears_missing predictionsBase
## Min.      :0.00000    Min.      :0.00000    Min.      :0.00003
## 1st Qu.:0.00000    1st Qu.:0.00000    1st Qu.:0.06169
## Median :0.00000    Median :0.00000    Median :0.10240
## Mean   :0.09477    Mean   :0.02465    Mean   :0.10831
## 3rd Qu.:0.00000    3rd Qu.:0.00000    3rd Qu.:0.14452
## Max.    :1.00000    Max.    :1.00000    Max.    :0.42429
## NA's    :1137
```

The isPrediction column was added correctly. The cutoff calculation marks roughly the top 11% highest-risk loans as predicted defaults.

```
validRows = !is.na(loanTest$predictionsBase)
```

```
tneg = sum((loanTest$isPrediction == 0) & (loanTest$isLoanDefault == 0) & validRows)
fneg = sum((loanTest$isPrediction == 0) & (loanTest$isLoanDefault == 1) & validRows)
fpos = sum((loanTest$isPrediction == 1) & (loanTest$isLoanDefault == 0) & validRows)
tpos = sum((loanTest$isPrediction == 1) & (loanTest$isLoanDefault == 1) & validRows)
```

```
tneg
```

```
## [1] 6848
```

```
fneg
```

```
## [1] 753
```

```
fpos
```

```
## [1] 760
```

```
tpos
```

```
## [1] 199
```

```
library(caret)
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
## lift
```

```
library(e1071)
```

```
##  
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':  
##  
## element
```

```
confusionMatrix(data = as.factor(loanTest$isprediction), reference = as.factor(loanTest$isLoanDefault))
```

```
## Confusion Matrix and Statistics  
##  
##           Reference  
## Prediction    0    1  
##           0 6848  753  
##           1  760  199  
##  
##           Accuracy : 0.8232  
##           95% CI : (0.815, 0.8313)  
##           No Information Rate : 0.8888  
##           P-Value [Acc > NIR] : 1.0000  
##  
##           Kappa : 0.1088  
##  
##           McNemar's Test P-Value : 0.8774  
##  
##           Sensitivity : 0.9001  
##           Specificity : 0.2090  
##           Pos Pred Value : 0.9009  
##           Neg Pred Value : 0.2075  
##           Prevalence : 0.8888  
##           Detection Rate : 0.8000  
##           Detection Prevalence : 0.8880  
##           Balanced Accuracy : 0.5546  
##  
##           'Positive' Class : 0  
##
```

```
confusionMatrix(data = as.factor(as.numeric(cutoff < loanTest$predictionsBase)),  
                reference = as.factor(loanTest$isLoanDefault))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 6849  753
##           1  759  199
##
##           Accuracy : 0.8234
##           95% CI : (0.8151, 0.8314)
##           No Information Rate : 0.8888
##           P-Value [Acc > NIR] : 1.0000
##
##           Kappa : 0.109
##
## Mcnemar's Test P-Value : 0.8977
##
##           Sensitivity : 0.9002
##           Specificity : 0.2090
##           Pos Pred Value : 0.9009
##           Neg Pred Value : 0.2077
##           Prevalence : 0.8888
##           Detection Rate : 0.8001
##           Detection Prevalence : 0.8881
##           Balanced Accuracy : 0.5546
##
##           'Positive' Class : 0
##
```

```
cutoff_higher = cutoff * 1.10
```

```
confusionMatrix(
  data = as.factor(as.numeric(cutoff_higher < loanTest$predictionsBase)),
  reference = as.factor(loanTest$isLoanDefault)
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 7196  827
##           1  412  125
##
##           Accuracy : 0.8553
##           95% CI : (0.8476, 0.8626)
##           No Information Rate : 0.8888
##           P-Value [Acc > NIR] : 1
##
##           Kappa : 0.0953
##
##           McNemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.9458
##           Specificity : 0.1313
##           Pos Pred Value : 0.8969
##           Neg Pred Value : 0.2328
##           Prevalence : 0.8888
##           Detection Rate : 0.8407
##           Detection Prevalence : 0.9373
##           Balanced Accuracy : 0.5386
##
##           'Positive' Class : 0
##
```

```
# Accuracy improved from .824 to .855
# Sensitivity improved to .945
# Specificity decreased to 0.1313
```

```
lowerCutoff = cutoff * 0.9

confusionMatrix(
  data = as.factor(as.numeric(lowerCutoff < loanTest$predictionsBase)),
  reference = as.factor(loanTest$isLoanDefault)
)
```



```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 6330  647
##           1 1278  305
##
##           Accuracy : 0.7751
##           95% CI : (0.7661, 0.7839)
##           No Information Rate : 0.8888
##           P-Value [Acc > NIR] : 1
##
##           Kappa : 0.1181
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.8320
##           Specificity : 0.3204
##           Pos Pred Value : 0.9073
##           Neg Pred Value : 0.1927
##           Prevalence : 0.8888
##           Detection Rate : 0.7395
##           Detection Prevalence : 0.8151
##           Balanced Accuracy : 0.5762
##
##           'Positive' Class : 0
##
```

```
# Accuracy decreased to .77
# Sensitivity decreased to .83
# Specificity increased to .32
```

```
summary(loanTest$predictionsBase)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
## 0.00003 0.06169 0.10240 0.10831 0.14452 0.42429    1137
```

```
confusionMatrix(data = as.factor(as.numeric(0.42 < loanTest$predictionsBase)),
                 reference = as.factor(loanTest$isLoanDefault))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 7607  951
##           1     1     1
##
##           Accuracy : 0.8888
##           95% CI : (0.8819, 0.8954)
##           No Information Rate : 0.8888
##           P-Value [Acc > NIR] : 0.5086
##
##           Kappa : 0.0016
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.99987
##           Specificity : 0.00105
##           Pos Pred Value : 0.88888
##           Neg Pred Value : 0.50000
##           Prevalence : 0.88879
##           Detection Rate : 0.88867
##           Detection Prevalence : 0.99977
##           Balanced Accuracy : 0.50046
##
##           'Positive' Class : 0
##
```

Maximizing accuracy can be misleading because most loans are good loans. A model that predicts 0 almost every time will still look accurate, even if it misses all the bad loans. High accuracy alone does not indicate the model is actually useful for detecting defaults.

```
glmTop5 = glm(isLoanDefault ~ incomeAnnual + creditGrade + interestRate +
              employmentYears + loanAmount,
              family = "binomial",
              data = loanTrain)

summary(glmTop5)
```

```
##
## Call:
## glm(formula = isLoanDefault ~ incomeAnnual + creditGrade + interestRate +
##       employmentYears + loanAmount, family = "binomial", data = loanTrain)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.045e+00  1.983e-01 -15.357  < 2e-16 ***
## incomeAnnual  -5.928e-06  8.186e-07  -7.242  4.43e-13 ***
## creditGradeB    3.901e-01  1.165e-01   3.349  0.000812 ***
## creditGradeC    6.308e-01  1.692e-01   3.729  0.000192 ***
## creditGradeD    7.315e-01  2.144e-01   3.411  0.000646 ***
## creditGradeE    7.830e-01  2.681e-01   2.921  0.003493 **
## creditGradeF    1.259e+00  3.534e-01   3.564  0.000365 ***
## creditGradeG    1.221e+00  4.888e-01   2.499  0.012457 *
## interestRate    7.249e-02  2.474e-02   2.930  0.003390 **
## employmentYears  9.461e-03  3.740e-03   2.530  0.011413 *
## loanAmount     -1.618e-06  4.482e-06  -0.361  0.718128
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 11658  on 17009  degrees of freedom
## Residual deviance: 11175  on 16999  degrees of freedom
## (2384 observations deleted due to missingness)
## AIC: 11197
##
## Number of Fisher Scoring iterations: 5
```

```
glmTop3 = glm(isLoanDefault ~ incomeAnnual + creditGrade + interestRate,
              family = "binomial",
              data = loanTrain)

summary(glmTop3)
```

```
##
## Call:
## glm(formula = isLoanDefault ~ incomeAnnual + creditGrade + interestRate,
##      family = "binomial", data = loanTrain)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -2.926e+00  1.913e-01 -15.296  < 2e-16 ***
## incomeAnnual -6.332e-06  7.293e-07  -8.681  < 2e-16 ***
## creditGradeB  3.804e-01  1.131e-01   3.363 0.000771 ***
## creditGradeC  6.181e-01  1.651e-01   3.745 0.000181 ***
## creditGradeD  6.927e-01  2.094e-01   3.307 0.000942 ***
## creditGradeE  7.459e-01  2.617e-01   2.850 0.004367 **
## creditGradeF  1.178e+00  3.474e-01   3.393 0.000692 ***
## creditGradeG  1.185e+00  4.826e-01   2.455 0.014096 *
## interestRate  7.182e-02  2.412e-02   2.978 0.002899 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 12224  on 17536  degrees of freedom
## Residual deviance: 11725  on 17528  degrees of freedom
## (1857 observations deleted due to missingness)
## AIC: 11743
##
## Number of Fisher Scoring iterations: 5
```

Based on AIC scores, glmTop5 is the best choice. Its AIC of approximately 11197 is lower than both glmBase (~11200) and glmTop3 (~11743). A lower AIC indicates a better balance between model fit and complexity. glmTop5 includes enough meaningful predictors to improve fit without overfitting.